TP Noté J4 - ARS Occitanie

TP NOTÉ — Data Platform Santé Publique

IPSSI Montpellier · Master 2 Data Engineering · Formation Apache Airflow

Champ	Valeur
Module	Orchestration de données avec Apache Airflow
Type	TP Noté — Évaluation individuelle
Durée	4 heures
Technologies	Docker Compose, CeleryExecutor, PostgreSQL, Airflow 2.8
Niveau	M2 — Data Engineering
Établissement	IPSSI Montpellier

> **AVERTISSEMENT** : ce TP est un travail strictement individuel. Toute communication avec un autre étudiant pendant la séance, tout partage de code ou de solution sera considéré comme une fraude et entraînera la note de 0 pour les deux parties concernées. L'utilisation d'outils d'IA générative (ChatGPT, Copilot, Claude, etc.) est interdite pendant l'examen. Seule la documentation officielle listée en section 9 est autorisée.

Table des matières

1. Contexte professionnel
2. Architecture technique cible
3. Setup Docker Compose
4. Données Santé Publique France — SurSaUD
5. Schéma de base de données PostgreSQL
6. Architecture du DAG `ars_epidemio_dag`
7. Travail demandé — 10 étapes
8. Livrables attendus
9. Contraintes techniques obligatoires
10. Ressources autorisées

---

1. Contexte professionnel

Mission

Vous êtes Lead Data Engineer à l'**Agence Régionale de Santé (ARS) Occitanie**, basée à Montpellier. L'ARS Occitanie coordonne un réseau de **150 médecins sentinelles** répartis sur les 13 départements de la région (09, 11, 12, 30, 31, 32, 34, 46, 48, 65, 66, 81, 82). Ces médecins transmettent chaque semaine, via le réseau national **SurSaUD** (Surveillance Sanitaire des Urgences et des Décès), le nombre de passages aux urgences et de consultations SOS Médecins pour plusieurs syndromes : grippe, gastro-entérite aiguë, syndrome grippal, bronchiolite (nourrissons), covid-19.

Chaque semaine, le service épidémiologie de l'ARS doit :

1. **Collecter** les données brutes de SurSaUD via l'API Santé Publique France
2. **Agréger** les cas par pathologie, par département et par semaine épidémiologique (format ISO : `YYYY-SXX`)
3. **Calculer** des indicateurs statistiques permettant de détecter une tendance anormale : taux d'incidence pour 100 000 habitants, z-score par rapport aux mêmes semaines sur les 5 dernières années, estimation du R0 par modèle SIR simplifié
4. **Alerter** les décideurs régionaux 48 heures avant que les seuils officiels nationaux ne soient dépassés, en croisant les données syndromiques avec les données météorologiques (température, humidité — facteurs aggravants connus)
5. **Archiver** les données brutes et les rapports dans un volume Docker persistant pendant une durée minimale de 5 ans, conformément aux obligations légales relatives aux données de santé publique

Problématique technique

L'ancienne infrastructure reposait sur des scripts Bash planifiés via cron et une base MySQL locale. Les limites sont atteintes :

- **Résilience nulle** : si la machine tombe, le pipeline s'arrête
- **Scalabilité absente** : les calculs statistiques sur 5 ans de données prennent 3 à 4 heures sur un seul cœur
- **Traçabilité insuffisante** : impossible de rejouer un pipeline pour une semaine passée
- **RGPD non satisfait** : les données de santé doivent être cloisonnées, chiffrées, accessibles uniquement aux personnes habilitées

Stack technique cible

Vous devez concevoir et déployer une **data platform** reposant sur :

- **Docker Compose** pour l'orchestration des services : chaque service tourne dans un conteneur isolé, les données sensibles ne persistent que dans des volumes nommés
- **Apache Airflow 2.8 avec CeleryExecutor** pour l'orchestration déclarative des pipelines — chaque tâche s'exécute dans un worker Celery dédié
- **DockerOperator** pour les tâches de traitement intensif (calculs statistiques, appels API) dans des conteneurs dédiés avec leurs propres dépendances Python
- **Volume Docker nommé** pour le stockage des archives brutes et des rapports, avec partitionnement temporel (`/data/ars/raw/<annee>/<semaine>/`)

- **PostgreSQL** pour les données opérationnelles : référentiels géographiques, indicateurs calculés, historique des alertes et des rapports générés

Contraintes RGPD et sécurité

Les données manipulées sont des données de **santé publique agrégées** (non nominatives au sens strict), mais leur croisement à l'échelle départementale peut permettre des ré-identifications dans les petits départements (Lozère — 48, Ariège — 09). À ce titre :

- Toutes les données doivent transiter via des connexions sécurisées (réseau Docker interne)
- Les connexions aux bases de données sont stockées dans les **Connections Airflow**, jamais en dur dans le code
- Les logs Airflow ne doivent pas contenir de données brutes de santé
- Le schéma PostgreSQL doit inclure des colonnes `created_at` et `updated_at` pour la traçabilité des modifications

---

2. Architecture technique cible

Schéma global

```
+-------------------------------------------------------------------
--+
|              INFRASTRUCTURE DOCKER COMPOSE
|
+-------------------------------------------------------------------
--+
|
|
|  +--------------+      +------------------+      +----------------+
|
|  |   Navigateur |----->| airflow-webserver|      |     flower     |
|
|  |   (port 8080)|      |   (port 8080)    |      |  (port 5555)   |
|
|  +--------------+      +------------------+      +-------+--------+
|
|                                  |                        |
|
|                                  v                        |
|
|                        +-------------------+              |
|
|                        | airflow-scheduler |              |
|
|                        +-------------------+              |
|
|                                  |                        |
|
```

```
|             +-------------------+-------------------+   |
|
|             |                                       |   |
|
|             v                                       v   v
|
|   +--------------------+                +--------------------+
|
|   |   airflow-worker   |                |   airflow-worker   |
|
|   |   (Celery worker)  |                |   (Celery worker)  |
|
|   +--------------------+                +--------------------+
|
|             |                                       |
|
|             +-------------------+-------------------+
|
|                                 |
|
|             +-------------------+-------------------+
|
|             |                   |                   |
|
|             v                   v                   v
|
|   +-----------------+  +----------------+  +----------------+     |
|   |      redis      |  |    postgres    |  |  postgres-ars  |     |
|   |  (broker Celery) |  | (métadonnées  |  | (données ARS)  |     |
|   |   port 6379     |  |   Airflow)     |  |  port 5433     |     |
|   +-----------------+  +----------------+  +----------------+     |
|
|
|             +-----------------------------------------+
|
|             |          Volume Docker nommé            |
|
|             |    ars-data:/data/ars/raw/<annee>/<sem>/  |
|
|             |    ars-data:/data/ars/rapports/         |
|
|             +-----------------------------------------+
|
|
|
|             +-----------------------------------------+
|
|             |         API Externes                    |
|
|             |  SurSaUD (data.gouv.fr)                 |
|
```

```
|              |  Open-Meteo (bonus)                       |
|
|              +-------------------------------------------+
|
+-------------------------------------------------------------------
--+
```

Flux de données bout en bout

```
[API SurSaUD]
      |
      | HTTP GET (JSON)
      v
[PythonOperator collecte] --------> [Volume /data/ars/raw/YYYY/SXX/]
      |                                     archive long terme
      | données brutes
      v
[PythonOperator calcul]
      | - taux incidence
      | - z-score vs N-1..N-5
      | - classification NORMAL/ALERTE/URGENCE
      v
[PostgreSQL ars_epidemio]
      | donnees_hebdomadaires
      | indicateurs_epidemiques
      v
[BranchPythonOperator]
      |-> URGENCE -> declencher_alerte_ars()
      |-> ALERTE  -> bulletin_surveillance()
      |-> NORMAL  -> confirmer_situation_normale()
              |
              v
      [generer_rapport_json] -------> [Volume /data/ars/rapports/]
```

---

3. Setup Docker Compose

3.1 Structure du projet

Créez la structure de répertoires suivante :

```
ars-epidemio/
├── dags/
│   ├── ars_epidemio_dag.py
│   └── sql/
│       └── init_ars_epidemio.sql
├── scripts/
│   ├── collecte_sursaud.py
│   └── calcul_indicateurs.py
```

```
├── docker-compose.yaml
├── .env
└── output/
    └── (rapports JSON générés ici)
```

3.2 Fichier docker-compose.yaml

```yaml
# docker-compose.yaml — Stack Airflow ARS Occitanie
# Airflow CeleryExecutor + Redis + PostgreSQL x2 + Volume persistant

x-airflow-common: &airflow-common
  image: apache/airflow:2.8.0
  environment: &airflow-common-env
    AIRFLOW__CORE__EXECUTOR: CeleryExecutor
    AIRFLOW__DATABASE__SQL_ALCHEMY_CONN:
        postgresql+psycopg2://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__RESULT_BACKEND:
        db+postgresql://airflow:airflow@postgres/airflow
    AIRFLOW__CELERY__BROKER_URL: redis://:@redis:6379/0
    AIRFLOW__CORE__FERNET_KEY: ''
    AIRFLOW__CORE__DAGS_ARE_PAUSED_AT_CREATION: 'true'
    AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
    AIRFLOW__API__AUTH_BACKENDS:
        'airflow.api.auth.backend.basic_auth,airflow.api.auth.backend.session'
    AIRFLOW__SCHEDULER__ENABLE_HEALTH_CHECK: 'true'
    AIRFLOW_CONN_POSTGRES_ARS:
        postgresql://postgres:postgres@postgres-ars:5432/ars_epidemio
  volumes:
    - ./dags:/opt/airflow/dags
    - ./scripts:/opt/airflow/scripts
    - airflow-logs:/opt/airflow/logs
    - ars-data:/data/ars
  user: "${AIRFLOW_UID:-50000}:0"
  depends_on: &airflow-common-depends-on
    redis:
      condition: service_healthy
    postgres:
      condition: service_healthy

services:
  postgres:
    image: postgres:13
    environment:
      POSTGRES_USER: airflow
      POSTGRES_PASSWORD: airflow
      POSTGRES_DB: airflow
    volumes:
      - postgres-db-volume:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD", "pg_isready", "-U", "airflow"]
      interval: 10s
      retries: 5
```

```yaml
    start_period: 5s
  restart: always

postgres-ars:
  image: postgres:13
  environment:
    POSTGRES_USER: postgres
    POSTGRES_PASSWORD: postgres
    POSTGRES_DB: ars_epidemio
  ports:
    - "5433:5432"
  volumes:
    - postgres-ars-volume:/var/lib/postgresql/data
  healthcheck:
    test: ["CMD", "pg_isready", "-U", "postgres"]
    interval: 10s
    retries: 5
  restart: always

redis:
  image: redis:latest
  expose:
    - 6379
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 10s
    timeout: 30s
    retries: 50
  restart: always

airflow-webserver:
  <<: *airflow-common
  command: webserver
  ports:
    - "8080:8080"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:8080/health"]
    interval: 30s
    timeout: 10s
    retries: 5
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

airflow-scheduler:
  <<: *airflow-common
  command: scheduler
  healthcheck:
```

```yaml
    test: ["CMD-SHELL", 'airflow jobs check --job-type SchedulerJob
      --hostname "$${HOSTNAME}"']
    interval: 30s
    timeout: 10s
    retries: 5
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

airflow-worker:
  <<: *airflow-common
  command: celery worker
  healthcheck:
    test:
      - "CMD-SHELL"
      - 'celery --app airflow.executors.celery_executor.app inspect
      ping -d "celery@$${HOSTNAME}"'
    interval: 30s
    timeout: 10s
    retries: 5
  environment:
    <<: *airflow-common-env
    DUMB_INIT_SETSID: "0"
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

flower:
  <<: *airflow-common
  command: celery flower
  ports:
    - "5555:5555"
  healthcheck:
    test: ["CMD", "curl", "--fail", "http://localhost:5555/"]
    interval: 30s
    timeout: 10s
    retries: 5
  restart: always
  depends_on:
    <<: *airflow-common-depends-on
    airflow-init:
      condition: service_completed_successfully

airflow-init:
  <<: *airflow-common
  entrypoint: /bin/bash
  command:
    - -c
```

```
      - |
        airflow db init
        airflow users create \
          --username admin \
          --firstname Admin \
          --lastname User \
          --role Admin \
          --email admin@ars-occitanie.fr \
          --password admin
    restart: on-failure
    depends_on:
      <<: *airflow-common-depends-on

volumes:
  postgres-db-volume:
  postgres-ars-volume:
  airflow-logs:
  ars-data:
```

3.3 Démarrage de la stack

```
# Créer le fichier .env
echo "AIRFLOW_UID=$(id -u)" > .env

# Démarrer tous les services
docker compose up -d

# Vérifier que tous les services sont Running
docker compose ps

# Résultat attendu :
# NAME                    STATUS
# ars-epidemio-postgres-1           running
# ars-epidemio-postgres-ars-1       running
# ars-epidemio-redis-1              running
# ars-epidemio-airflow-webserver-1  running
# ars-epidemio-airflow-scheduler-1  running
# ars-epidemio-airflow-worker-1     running
# ars-epidemio-flower-1             running
```

3.4 Accès aux interfaces

- **Airflow UI** : http://localhost:8080 (admin / admin)
- **Flower** : http://localhost:5555
- **PostgreSQL ARS** : localhost:5433 (postgres / postgres / ars_epidemio)

3.5 Accès aux logs et debugging

```
# Voir les logs d'un service
docker compose logs airflow-scheduler -f
```

```
# Voir les logs d'un worker
docker compose logs airflow-worker -f

# Entrer dans un conteneur pour déboguer
docker compose exec airflow-worker bash

# Vérifier le volume de données
docker compose exec airflow-worker ls -la /data/ars/
```

---

4. Données — Indicateurs Avancés Sanitaires (IAS®)

4.1 Présentation des datasets

L'**Indicateur Avancé Sanitaire (IAS®)** est produit quotidiennement par **OpenHealth / CELTIPHARM** à partir des données de vente en pharmacie (antigrippaux, antiémétiques) couplées aux données de médecins sentinelles. Il permet de détecter l'activité épidémique **2 à 5 jours avant** les systèmes de surveillance classiques.

Deux datasets distincts sont disponibles en open data sur data.gouv.fr (licence ODbL) :

Syndrome	Page dataset	URL CSV régionaux (
Syndrome Grippal	https://www.data.gouv.fr/fr/datasets/indicateur-avance-sanitaire-ias-syndrome-grippal/	https://www.data.gouv.fr/ 7a97-46b3-a93c-35a4710
Gastro-Entérite	https://www.data.gouv.fr/fr/datasets/indicateur-avance-sanitaire-ias-gastro-enterite/	https://www.data.gouv.fr/ 4ebf-4af5-b0dc-9867bb1e

> Ces fichiers sont mis à jour **quotidiennement** et sont accessibles sans authentification.

4.2 Format et structure des données

Les deux CSV ont exactement la même structure. Points d'attention :

- **Séparateur de colonnes** : `;` (point-virgule)
- **Séparateur décimal** : `,` (virgule — format français)
- **Encoding** : UTF-8

Colonnes principales :

Colonne	Type	Description
`PERIODE`	string	Date au format `DD-MM-YYYY`
`Loc_Reg11` … `Loc_Reg94`	float	Valeur IAS pour chaque région (code INSEE)
`Loc_Reg76`	float	**Valeur IAS pour l'Occitanie** (région 76) — colonne cible du TP

Colonne	Type	Description
`Sais_2023_2024`	float	Valeur IAS même date, saison 2023-2024 (référence historique N-1)
`Sais_2022_2023`	float	Valeur IAS même date, saison 2022-2023 (N-2)
`Sais_2021_2022`	float	N-3
`Sais_2020_2021`	float	N-4
`Sais_2019_2020`	float	N-5
`MIN_Saison`	float	Seuil bas : valeur à partir de laquelle une épidémie est déclarée
`MAX_Saison`	float	Seuil haut : valeur correspondant à un pic épidémique intense
`Senti`	float	Données réseau Sentinelles (souvent NA)

Extrait des premières lignes (CSV Grippe) :

```
PERIODE;Senti;Sais_2023_2024;Sais_2022_2023;...;Loc_Reg76;...;MIN_Sais
01-01-2025;NA;45.2;38.7;...;52.4;...;80.0;210.0
02-01-2025;NA;46.1;40.2;...;54.1;...;80.0;210.0
```

Agrégation hebdomadaire : les données sont quotidiennes. Pour obtenir une valeur par **semaine ISO** (format `YYYY-SXX`), vous devez regrouper les jours de la semaine concernée et calculer la **moyenne** de `Loc_Reg76` sur ces jours.

4.3 Seuils d'alerte épidémique

Les colonnes `MIN_Saison` et `MAX_Saison` du dataset fournissent directement les seuils épidémiques définis par OpenHealth. La classification est la suivante :

Niveau	Critère IAS vs seuils	Critère z-score	Action
NORMAL	`valeur_ias < MIN_Saison`	$z < 1.5$	Surveillance standard
ALERTE	`MIN_Saison ≤ valeur_ias < MAX_Saison`	$1.5 \leq z < 3.0$	Bulletin de surveillance hebdomadaire
URGENCE	`valeur_ias ≥ MAX_Saison`	$z \geq 3.0$	Alerte ARS + mobilisation renforcée

> La classification finale retient le **niveau le plus sévère** entre les deux critères (IAS vs seuils et z-score vs historique).

4.4 Indicateurs à calculer

Valeur IAS hebdomadaire pour Occitanie :

```
valeur_ias_semaine = moyenne(Loc_Reg76) sur les jours de la semaine
ISO
```

Z-score épidémique (méthode OMS adaptée) :

```
z = (valeur_ias_semaine - moyenne_historique) / ecart_type_historique
```

où `moyenne_historique` et `ecart_type_historique` sont calculés sur les colonnes `Sais_2019_2020` à `Sais_2023_2024` **pour les mêmes jours de la semaine** (minimum 3 saisons requises).

Estimation du R0 simplifié (modèle SIR) :

```
R0_estimé = 1 + (taux_croissance_hebdomadaire *
durée_infectieuse_jours / 7)
```

avec `durée_infectieuse_jours = 5` pour la grippe, `3` pour la gastro-entérite, où `taux_croissance_hebdomadaire` est calculé sur les 4 dernières valeurs IAS hebdomadaires disponibles en base.

---

5. Schéma de base de données PostgreSQL

La base de données `ars_epidemio` doit être créée sur le service `postgres-ars`. Voici le schéma complet :

```sql
-- ============================================================
-- BASE DE DONNÉES : ars_epidemio
-- Schéma pour la surveillance épidémiologique ARS Occitanie
-- ============================================================

-- Table des syndromes surveillés
CREATE TABLE IF NOT EXISTS syndromes (
    id                    SERIAL PRIMARY KEY,
    code                  VARCHAR(20) UNIQUE NOT NULL,
    libelle               VARCHAR(100) NOT NULL,
    description           TEXT,
    duree_infectieuse_jours INTEGER DEFAULT 5,
    actif                 BOOLEAN DEFAULT TRUE,
    created_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    updated_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP
);

-- Table des départements (référentiel géographique)
CREATE TABLE IF NOT EXISTS departements (
    id                    SERIAL PRIMARY KEY,
```

```sql
    code_dept             VARCHAR(3) UNIQUE NOT NULL,
    nom                   VARCHAR(100) NOT NULL,
    code_region           VARCHAR(3) NOT NULL,
    nom_region            VARCHAR(100) NOT NULL,
    population            INTEGER NOT NULL,
    superficie_km2        FLOAT,
    chef_lieu             VARCHAR(100),
    created_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    updated_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP
);

CREATE INDEX IF NOT EXISTS idx_departements_region
    ON departements (code_region);

-- Table des données hebdomadaires agrégées (IAS® Occitanie)
-- Une ligne par (semaine, syndrome) — agrégation hebdomadaire des
        données quotidiennes IAS
CREATE TABLE IF NOT EXISTS donnees_hebdomadaires (
    id                    SERIAL PRIMARY KEY,
    semaine               VARCHAR(8) NOT NULL,  -- format YYYY-SXX
    syndrome              VARCHAR(20) NOT NULL REFERENCES
        syndromes(code),
    valeur_ias            FLOAT NOT NULL,        -- moyenne IAS
        Occitanie (Loc_Reg76) sur la semaine
    seuil_min_saison      FLOAT,                 -- MIN_Saison du
        dataset (seuil épidémique bas)
    seuil_max_saison      FLOAT,                 -- MAX_Saison du
        dataset (seuil pic épidémique)
    nb_jours_donnees      INTEGER DEFAULT 0,     -- nombre de jours
        disponibles pour l'agrégation
    annee                 INTEGER GENERATED ALWAYS AS
        (CAST(LEFT(semaine, 4) AS INTEGER)) STORED,
    numero_semaine        INTEGER GENERATED ALWAYS AS
        (CAST(RIGHT(semaine, 2) AS INTEGER)) STORED,
    created_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    updated_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    CONSTRAINT uq_donnees_hbd UNIQUE (semaine, syndrome)
);

CREATE INDEX IF NOT EXISTS idx_donnees_hbd_semaine
    ON donnees_hebdomadaires (semaine);
CREATE INDEX IF NOT EXISTS idx_donnees_hbd_syndrome_annee
    ON donnees_hebdomadaires (syndrome, annee, numero_semaine);

-- Table des indicateurs épidémiques calculés (niveau régional
        Occitanie)
CREATE TABLE IF NOT EXISTS indicateurs_epidemiques (
    id                    SERIAL PRIMARY KEY,
    semaine               VARCHAR(8) NOT NULL,
```

```sql
    syndrome              VARCHAR(20) NOT NULL REFERENCES
        syndromes(code),
    valeur_ias            FLOAT,       -- valeur IAS hebdomadaire
        Occitanie
    z_score               FLOAT,       -- par rapport aux N-1..N-5
        mêmes semaines
    r0_estime             FLOAT,       -- reproduction number estimé
        (modèle SIR simplifié)
    nb_saisons_reference  INTEGER,     -- nombre de saisons
        historiques utilisées pour le z-score
    statut                VARCHAR(10) CHECK (statut IN ('NORMAL',
        'ALERTE', 'URGENCE')),
    statut_ias            VARCHAR(10) CHECK (statut_ias IN ('NORMAL',
        'ALERTE', 'URGENCE')),
    statut_zscore         VARCHAR(10) CHECK (statut_zscore IN
        ('NORMAL', 'ALERTE', 'URGENCE')),
    commentaire           TEXT,
    calcule_le            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    created_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    updated_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    CONSTRAINT uq_indicateurs UNIQUE (semaine, syndrome)
);

CREATE INDEX IF NOT EXISTS idx_indicateurs_statut
    ON indicateurs_epidemiques (statut, semaine);
CREATE INDEX IF NOT EXISTS idx_indicateurs_semaine
    ON indicateurs_epidemiques (semaine);

-- Table des rapports ARS générés
CREATE TABLE IF NOT EXISTS rapports_ars (
    id                    SERIAL PRIMARY KEY,
    semaine               VARCHAR(8) NOT NULL,
    situation_globale     VARCHAR(10) CHECK (situation_globale IN
        ('NORMAL', 'ALERTE', 'URGENCE')),
    nb_depts_alerte       INTEGER DEFAULT 0,
    nb_depts_urgence      INTEGER DEFAULT 0,
    rapport_json          JSONB,       -- contenu complet du rapport
    chemin_local          VARCHAR(500),  -- chemin dans le volume
        Docker
    genere_le             TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    genere_par            VARCHAR(100) DEFAULT 'ars_epidemio_dag',
    created_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    updated_at            TIMESTAMP WITH TIME ZONE DEFAULT
        CURRENT_TIMESTAMP,
    CONSTRAINT uq_rapport_semaine UNIQUE (semaine)
);

-- ============================================================
-- DONNÉES DE RÉFÉRENCE
-- ============================================================
```

```sql
-- Insertion des syndromes surveillés
INSERT INTO syndromes (code, libelle, description,
        duree_infectieuse_jours) VALUES
    ('GRIPPE',   'Grippe',                'Syndrome grippal confirmé
        ou probable', 5),
    ('GEA',      'Gastro-entérite aiguë', 'Diarrhées et/ou
        vomissements aigus',    3),
    ('SG',       'Syndrome grippal',      'Fièvre + toux ou myalgies
        sans confirmation', 5),
    ('BRONCHIO', 'Bronchiolite',          'Bronchiolite du nourrisson
        (< 2 ans)',   7),
    ('COVID19',  'Covid-19',              'Syndrome Covid-19 confirmé
        ou probable', 7)
ON CONFLICT (code) DO NOTHING;

-- Insertion des 13 départements d'Occitanie
INSERT INTO departements (code_dept, nom, code_region, nom_region,
        population, chef_lieu) VALUES
    ('09', 'Ariège',            '76', 'Occitanie', 153153,  'Foix'),
    ('11', 'Aude',              '76', 'Occitanie', 375438,
        'Carcassonne'),
    ('12', 'Aveyron',           '76', 'Occitanie', 279594,  'Rodez'),
    ('30', 'Gard',              '76', 'Occitanie', 750294,  'Nîmes'),
    ('31', 'Haute-Garonne',     '76', 'Occitanie', 1432680,
        'Toulouse'),
    ('32', 'Gers',              '76', 'Occitanie', 191283,  'Auch'),
    ('34', 'Hérault',           '76', 'Occitanie', 1175623,
        'Montpellier'),
    ('46', 'Lot',               '76', 'Occitanie', 174208,
        'Cahors'),
    ('48', 'Lozère',            '76', 'Occitanie', 76601,   'Mende'),
    ('65', 'Hautes-Pyrénées',   '76', 'Occitanie', 228342,
        'Tarbes'),
    ('66', 'Pyrénées-Orientales','76', 'Occitanie', 479979,
        'Perpignan'),
    ('81', 'Tarn',              '76', 'Occitanie', 389521,  'Albi'),
    ('82', 'Tarn-et-Garonne',   '76', 'Occitanie', 261100,
        'Montauban')
ON CONFLICT (code_dept) DO NOTHING;

-- Trigger pour mettre à jour updated_at automatiquement
CREATE OR REPLACE FUNCTION update_updated_at_column()
RETURNS TRIGGER AS $$
BEGIN
    NEW.updated_at = CURRENT_TIMESTAMP;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER update_donnees_hbd_updated_at
    BEFORE UPDATE ON donnees_hebdomadaires
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

```
CREATE TRIGGER update_indicateurs_updated_at
    BEFORE UPDATE ON indicateurs_epidemiques
    FOR EACH ROW EXECUTE FUNCTION update_updated_at_column();
```

---

6. Architecture du DAG `ars_epidemio_dag`

6.1 Diagramme des tâches

```
DAG : ars_epidemio_dag
schedule_interval: "0 6 * * 1"  (tous les lundis à 6h)
catchup: True  |  max_active_runs: 1

[init_base_donnees] (PostgresOperator)
        |
        v
+-----------------------------------------------+
| TaskGroup : collecte                          |
|   [collecter_donnees_sursaud]                 |
|   (PythonOperator)                            |
|   -> télécharge données SPF                   |
|   -> stocke dans /data/ars/raw/               |
+-----------------------------------------------+
        |
        v
+-----------------------------------------------+
| TaskGroup : persistance_brute                 |
|   [archiver_local]                            |
|   (PythonOperator)                            |
|   -> écrit dans /data/ars/raw/YYYY/SXX/       |
|                                               |
|   [verifier_archive]                          |
|   (PythonOperator)                            |
|   -> vérifie que le fichier existe            |
+-----------------------------------------------+
        |
        v
+-----------------------------------------------+
| TaskGroup : traitement                        |
|   [calculer_indicateurs_epidemiques]          |
|   (PythonOperator)                            |
|   -> z-score, taux_incidence, R0 estimé       |
|   -> classification NORMAL / ALERTE / URGENCE |
+-----------------------------------------------+
        |
        v
+-----------------------------------------------+
| TaskGroup : persistance_operationnelle        |
|   [inserer_donnees_postgres]                  |
```

```
|   (PostgresOperator ou PythonOperator)          |
|   -> INSERT INTO donnees_hebdomadaires          |
|   -> INSERT INTO indicateurs_epidemiques        |
|   -> ON CONFLICT DO UPDATE (idempotence)        |
+-------------------------------------------------+
        |
        v
[evaluer_situation_epidemique] (BranchPythonOperator)
        |
   +----|----+--------+
   v         v        v
[declencher_ [envoyer_ [confirmer_
 alerte_ars]  bulletin_  situation_
(PythonOp)   surveill.] normale]
             (PythonOp) (PythonOp)
        |         |          |
        +----+----+----------+
             |
             v
[generer_rapport_hebdomadaire] (PythonOperator)
TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS
-> JSON structuré
-> sauvegarde /data/ars/rapports/
-> INSERT INTO rapports_ars
```

6.2 En-tête et paramètres du DAG

```python
from datetime import datetime, timedelta
from airflow import DAG
from airflow.utils.dates import days_ago

default_args = {
    "owner": "ars-occitanie",
    "depends_on_past": False,
    "email_on_failure": True,
    "email_on_retry": False,
    "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "execution_timeout": timedelta(hours=2),
}

with DAG(
    dag_id="ars_epidemio_dag",
    default_args=default_args,
    description="Pipeline surveillance épidémiologique ARS Occitanie",
    schedule_interval="0 6 * * 1",   # Tous les lundis à 6h UTC
    start_date=datetime(2024, 1, 1),
    catchup=True,
    max_active_runs=1,
    tags=["sante-publique", "epidemio", "docker-compose"],
```

```
) as dag:
    pass  # Implémentation dans les étapes suivantes
```

---

7. Travail demandé — 10 étapes

Etape 1 — Setup Docker Compose et vérification de l'environnement

Objectif : Démarrer la stack Docker Compose et vérifier que l'ensemble des services est opérationnel avant de commencer à coder.

Tâches :

1. Créer le fichier `docker-compose.yaml` fourni en section 3.2
2. Créer le fichier `.env` avec `AIRFLOW_UID`
3. Démarrer la stack avec `docker compose up -d`
4. Vérifier que tous les services sont en état `running` (`docker compose ps`)
5. Accéder à l'UI Airflow via http://localhost:8080 et vérifier qu'aucun DAG d'erreur n'est présent
6. Accéder à Flower via http://localhost:5555 et vérifier qu'au moins un worker est connecté

Livrables de cette étape :

- Screenshot de `docker compose ps` montrant tous les services en état `running`
- Screenshot de l'UI Airflow accessible à http://localhost:8080
- Screenshot de Flower montrant le(s) worker(s) actif(s)

Commandes attendues :

```
echo "AIRFLOW_UID=$(id -u)" > .env
docker compose up -d
docker compose ps
```

Etape 2 — Configuration des connexions Airflow et des Variables

Objectif : Configurer l'ensemble des connexions aux services et les variables de paramétrage du pipeline.

Connexions à créer dans l'UI Airflow (Admin > Connections) :

Conn Id	Type	Host	Port	Schema	Login	Passwo
`postgres_ars`	Postgres	`postgres-ars`	5432	`ars_epidemio`	`postgres`	`postgr`

Variables à créer dans l'UI Airflow (Admin > Variables) :

Clé	Valeur
`semaines_historique`	12

Clé	Valeur
seuil_alerte_incidence	150
seuil_urgence_incidence	500
seuil_alerte_zscore	1.5
seuil_urgence_zscore	3.0
departements_occitanie	["09","11","12","30","31","32","34","46","48'
syndromes_surveilles	["GRIPPE","GEA","SG","BRONCHIO","COVID19"]
archive_base_path	/data/ars

Vérification : Depuis un PythonOperator de test, vérifier que les connexions sont accessibles :

```
from airflow.hooks.base import BaseHook
from airflow.models import Variable

def verifier_connexions():
    conn_pg = BaseHook.get_connection("postgres_ars")
    depts = Variable.get("departements_occitanie",
        deserialize_json=True)
    print(f"PostgreSQL : {conn_pg.host}:
        {conn_pg.port}/{conn_pg.schema}")
    print(f"Départements configurés : {len(depts)}")
```

Etape 3 — Initialisation de la base de données PostgreSQL

Objectif : Créer toutes les tables et insérer les données de référence depuis le DAG.

Implémentation :

```
from airflow.providers.postgres.operators.postgres import
        PostgresOperator

init_base_donnees = PostgresOperator(
    task_id="init_base_donnees",
    postgres_conn_id="postgres_ars",
    sql="sql/init_ars_epidemio.sql",  # Fichier SQL séparé dans le
        dossier dags/sql/
    autocommit=True,
)
```

Le fichier `sql/init_ars_epidemio.sql` contient l'intégralité du schéma SQL présenté en section 5, avec les instructions `CREATE TABLE IF NOT EXISTS` et `INSERT ... ON CONFLICT DO NOTHING` pour garantir l'**idempotence** : le DAG peut être relancé sans erreur même si les tables existent déjà.

Vérification attendue :

```sql
-- Vérifier la création des tables
SELECT table_name,
        pg_size_pretty(pg_total_relation_size(table_name::regclass))
        AS taille
FROM information_schema.tables
WHERE table_schema = 'public'
ORDER BY table_name;

-- Vérifier les données de référence
SELECT code, libelle FROM syndromes;
SELECT code_dept, nom, population FROM departements ORDER BY
        code_dept;
```

Etape 4 — Implémentation de `collecter_donnees_sursaud`

Objectif : Créer une tâche Python qui télécharge les données SurSaUD pour la semaine en cours et les stocke dans `/data/ars/raw/`.

Script Python à implémenter (`scripts/collecte_ias.py`) :

```python
#!/usr/bin/env python3
"""
Script de collecte des données IAS® (Indicateurs Avancés Sanitaires)
Grippe et Gastro-entérite — OpenHealth / data.gouv.fr
"""
import csv
import io
import json
import logging
import os
from datetime import datetime, date
from typing import Optional

import requests

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# URLs directes des CSV régionaux sur data.gouv.fr
DATASETS_IAS = {
    "GRIPPE": "https://www.data.gouv.fr/api/1/datasets/r/35f46fbb-
        7a97-46b3-a93c-35a471033447",
    "GEA":    "https://www.data.gouv.fr/api/1/datasets/r/6c415be9-
        4ebf-4af5-b0dc-9867bb1ec0e3",
}

# Colonne Occitanie dans les CSV (région INSEE 76)
COL_OCCITANIE = "Loc_Reg76"


def get_semaine_iso(reference_date: Optional[date] = None) -> str:
```

```python
    """Retourne la semaine ISO au format YYYY-SXX."""
    if reference_date is None:
        reference_date = date.today()
    year, week, _ = reference_date.isocalendar()
    return f"{year}-S{week:02d}"


def telecharger_csv_ias(url: str) -> list[dict]:
    """
    Télécharge et parse un CSV IAS® depuis data.gouv.fr.
    Gère le séparateur ';' et le séparateur décimal ',' (format
        français).
    """
    logger.info(f"Téléchargement : {url}")
    response = requests.get(url, timeout=60)
    response.raise_for_status()

    # Décodage UTF-8
    content = response.content.decode("utf-8")

    # Parser le CSV (séparateur ';', décimal ',')
    reader = csv.DictReader(io.StringIO(content), delimiter=";")
    rows = []
    for row in reader:
        # Convertir les virgules décimales en points
        cleaned = {
            k: v.replace(",", ".") if v not in ("NA", "", None) else
        None
            for k, v in row.items()
        }
        rows.append(cleaned)
    logger.info(f"{len(rows)} lignes récupérées")
    return rows


def filtrer_semaine(
    rows: list[dict], semaine: str
) -> list[dict]:
    """
    Filtre les lignes correspondant à la semaine ISO demandée.
    Convertit PERIODE (DD-MM-YYYY) en numéro de semaine ISO.
    """
    annee_cible = int(semaine[:4])
    num_sem_cible = int(semaine[2:])  # après le 'S'

    filtered = []
    for row in rows:
        periode = row.get("PERIODE")
        if not periode:
            continue
        try:
            d = datetime.strptime(periode, "%d-%m-%Y").date()
```

```
            iso_year, iso_week, _ = d.isocalendar()
            if iso_year == annee_cible and iso_week == num_sem_cible:
                filtered.append(row)
        except ValueError:
            continue
    logger.info(f"{len(filtered)} jours pour la semaine {semaine}")
    return filtered


def agreger_semaine(rows: list[dict], syndrome: str, semaine: str) ->
        dict:
    """
    Agrège les lignes d'une semaine en une valeur IAS hebdomadaire.
    Calcule la moyenne de Loc_Reg76 et récupère les données
        historiques.
    """
    saisons_cols = [
        "Sais_2023_2024", "Sais_2022_2023", "Sais_2021_2022",
        "Sais_2020_2021", "Sais_2019_2020",
    ]

    valeurs_ias, min_saison_vals, max_saison_vals = [], [], []
    historique: dict[str, list[float]] = {col: [] for col in
        saisons_cols}

    for row in rows:
        val = row.get(COL_OCCITANIE)
        if val is not None:
            try:
                valeurs_ias.append(float(val))
            except ValueError:
                pass
        for col in saisons_cols:
            v = row.get(col)
            if v is not None:
                try:
                    historique[col].append(float(v))
                except ValueError:
                    pass
        for field, dest in [("MIN_Saison", min_saison_vals),
        ("MAX_Saison", max_saison_vals)]:
            v = row.get(field)
            if v is not None:
                try:
                    dest.append(float(v))
                except ValueError:
                    pass

    def safe_mean(lst: list[float]) -> Optional[float]:
        return round(sum(lst) / len(lst), 3) if lst else None

    return {
```

```
        "semaine":         semaine,
        "syndrome":        syndrome,
        "valeur_ias":      safe_mean(valeurs_ias),
        "seuil_min":       safe_mean(min_saison_vals),
        "seuil_max":       safe_mean(max_saison_vals),
        "nb_jours":        len(valeurs_ias),
        "historique":      {col: safe_mean(vals) for col, vals in
        historique.items()},
    }


def sauvegarder_donnees(donnees: dict, semaine: str, output_dir: str)
        -> str:
    """Sauvegarde les données IAS agrégées en JSON."""
    os.makedirs(output_dir, exist_ok=True)
    output_path = os.path.join(output_dir, f"ias_{semaine}.json")
    with open(output_path, "w", encoding="utf-8") as f:
        json.dump({
            "semaine":      semaine,
            "collecte_le":  datetime.utcnow().isoformat(),
            "source":       "IAS_OpenHealth_data.gouv.fr",
            "syndromes":    donnees,
        }, f, ensure_ascii=False, indent=2)
    logger.info(f"Données sauvegardées : {output_path}")
    return output_path


if __name__ == "__main__":
    semaine = os.environ.get("SEMAINE_CIBLE", get_semaine_iso())
    output_dir = os.environ.get("OUTPUT_DIR", "/data/ars/raw")

    resultats = {}
    for syndrome, url in DATASETS_IAS.items():
        rows_all = telecharger_csv_ias(url)
        rows_sem = filtrer_semaine(rows_all, semaine)
        resultats[syndrome] = agreger_semaine(rows_sem, syndrome,
        semaine)

    chemin = sauvegarder_donnees(resultats, semaine, output_dir)
    print(f"COLLECTE_OK:{chemin}")
```

Implémentation du PythonOperator dans le DAG :

```
from airflow.operators.python import PythonOperator
from airflow.models import Variable
import sys

def collecter_donnees_ias(**context) -> str:
    """Télécharge les CSV IAS® et retourne le chemin du fichier JSON
        créé."""
    semaine = (
        f"{context['execution_date'].year}-"
```

```
        f"S{context['execution_date'].isocalendar()[1]:02d}"
    )
    archive_path = Variable.get("archive_base_path",
        default_var="/data/ars")
    output_dir = f"{archive_path}/raw"

    sys.path.insert(0, "/opt/airflow/scripts")
    from collecte_ias import (
        DATASETS_IAS, telecharger_csv_ias, filtrer_semaine,
        agreger_semaine, sauvegarder_donnees
    )

    resultats = {}
    for syndrome, url in DATASETS_IAS.items():
        rows_all = telecharger_csv_ias(url)
        rows_sem = filtrer_semaine(rows_all, semaine)
        resultats[syndrome] = agreger_semaine(rows_sem, syndrome,
        semaine)

    return sauvegarder_donnees(resultats, semaine, output_dir)

collecter_sursaud = PythonOperator(
    task_id="collecter_donnees_sursaud",
    python_callable=collecter_donnees_ias,
    provide_context=True,
)
```

Etape 5 — Implémentation de `archiver_local` et `verifier_archive`

Objectif : Organiser les données brutes collectées dans le volume Docker avec un partitionnement par année et semaine, puis vérifier l'intégrité.

Convention de nommage :

```
/data/ars/raw/<annee>/<semaine>/sursaud_<semaine>.json
/data/ars/raw/<annee>/<semaine>/meteo_<semaine>.json  (bonus)

Exemple :
/data/ars/raw/2024/S04/sursaud_2024-S04.json
```

Implémentation :

```
import os, shutil
from airflow.operators.python import PythonOperator

def archiver_local(**context) -> str:
    """
    Organise le fichier brut dans la structure d'archivage
        partitionnée.
    """
    semaine = (
```

```python
        f"{context['execution_date'].year}-"
        f"S{context['execution_date'].isocalendar()[1]:02d}"
    )
    annee = semaine.split("-")[0]
    num_sem = semaine.split("-")[1]

    # Récupération du chemin depuis XCom
    chemin_source = context["task_instance"].xcom_pull(
        task_ids="collecte.collecter_donnees_sursaud"
    )

    archive_dir = f"/data/ars/raw/{annee}/{num_sem}"
    os.makedirs(archive_dir, exist_ok=True)
    chemin_dest = f"{archive_dir}/sursaud_{semaine}.json"
    shutil.copy2(chemin_source, chemin_dest)

    print(f"ARCHIVE_OK:{chemin_dest}")
    return chemin_dest

def verifier_archive(**context) -> bool:
    """Vérifie que le fichier d'archive existe et n'est pas vide."""
    semaine = (
        f"{context['execution_date'].year}-"
        f"S{context['execution_date'].isocalendar()[1]:02d}"
    )
    annee = semaine.split("-")[0]
    num_sem = semaine.split("-")[1]
    chemin = f"/data/ars/raw/{annee}/{num_sem}/sursaud_{semaine}.json"

    if not os.path.exists(chemin):
        raise FileNotFoundError(f"Archive manquante : {chemin}")
    taille = os.path.getsize(chemin)
    if taille == 0:
        raise ValueError(f"Archive vide : {chemin}")
    print(f"ARCHIVE_VALIDE:{chemin} ({taille} octets)")
    return True

archiver = PythonOperator(
    task_id="archiver_local",
    python_callable=archiver_local,
    provide_context=True,
)

verifier = PythonOperator(
    task_id="verifier_archive",
    python_callable=verifier_archive,
    provide_context=True,
)
```

Etape 6 — Implémentation de `calculer_indicateurs_epidemiques`

Objectif : Calculer le taux d'incidence, le z-score et la classification épidémique pour chaque département et syndrome.

Script Python à implémenter (`scripts/calcul_indicateurs.py`) :

```python
#!/usr/bin/env python3
"""
Calcul des indicateurs épidémiques IAS® — ARS Occitanie
"""
import json
import logging
import os
from typing import Optional

import numpy as np
import psycopg2

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)


def calculer_zscore(
    valeur_actuelle: float, historique: list[float]
) -> Optional[float]:
    """
    Calcule le z-score de la valeur IAS par rapport aux saisons
        historiques.
    Requiert au minimum 3 valeurs historiques.
    """
    valeurs_valides = [v for v in historique if v is not None]
    if len(valeurs_valides) < 3:
        logger.warning(f"Historique insuffisant
        ({len(valeurs_valides)} saisons)")
        return None
    moyenne = np.mean(valeurs_valides)
    ecart_type = np.std(valeurs_valides, ddof=1)
    if ecart_type == 0:
        return 0.0
    return float((valeur_actuelle - moyenne) / ecart_type)


def classifier_statut_ias(
    valeur_ias: float,
    seuil_min: Optional[float],
    seuil_max: Optional[float],
) -> str:
    """Classifie selon les seuils MIN/MAX du dataset IAS."""
    if seuil_max is not None and valeur_ias >= seuil_max:
        return "URGENCE"
    if seuil_min is not None and valeur_ias >= seuil_min:
```

```
        return "ALERTE"
    return "NORMAL"


def classifier_statut_zscore(
    z_score: Optional[float],
    seuil_alerte_z: float = 1.5,
    seuil_urgence_z: float = 3.0,
) -> str:
    """Classifie selon le z-score par rapport à l'historique des
        saisons."""
    if z_score is None:
        return "NORMAL"
    if z_score >= seuil_urgence_z:
        return "URGENCE"
    if z_score >= seuil_alerte_z:
        return "ALERTE"
    return "NORMAL"


def classifier_statut_final(statut_ias: str, statut_zscore: str) ->
        str:
    """Retient le niveau le plus sévère entre les deux critères."""
    if "URGENCE" in (statut_ias, statut_zscore):
        return "URGENCE"
    if "ALERTE" in (statut_ias, statut_zscore):
        return "ALERTE"
    return "NORMAL"


def calculer_r0_simplifie(
    series_hebdomadaire: list[float],
    duree_infectieuse: int = 5,
) -> Optional[float]:
    """Estimation du R0 par calcul du taux de croissance moyen sur les
        séries IAS."""
    series_valides = [v for v in series_hebdomadaire if v is not None
        and v > 0]
    if len(series_valides) < 2:
        return None
    croissances = [
        (series_valides[i] - series_valides[i - 1]) / series_valides[i
        - 1]
        for i in range(1, len(series_valides))
    ]
    if not croissances:
        return None
    return max(0.0, float(1 + np.mean(croissances) *
        (duree_infectieuse / 7)))
```

Etape 7 — Implémentation de `inserer_donnees_postgres`

Objectif : Lire les résultats du calcul et les persister dans PostgreSQL avec gestion des conflits (idempotence).

```python
from airflow.providers.postgres.hooks.postgres import PostgresHook
from airflow.operators.python import PythonOperator
import json, logging

logger = logging.getLogger(__name__)

def inserer_donnees_postgres(**context) -> None:
    """
    Insère les données hebdomadaires et les indicateurs dans
        PostgreSQL.
    Utilise ON CONFLICT DO UPDATE pour garantir l'idempotence.
    """
    semaine: str = context["templates_dict"]["semaine"]

    annee, num_sem = semaine.split("-")
    with
        open(f"/data/ars/raw/{annee}/{num_sem}/sursaud_{semaine}.json")
        as f:
        donnees_brutes = json.load(f)

    with open(f"/data/ars/indicateurs/indicateurs_{semaine}.json") as
        f:
        indicateurs = json.load(f)

    hook = PostgresHook(postgres_conn_id="postgres_ars")

    sql_donnees = """
        INSERT INTO donnees_hebdomadaires
            (code_dept, semaine, syndrome, nb_passages_urg,
        nb_consultations,
             nb_total_passages, taux_recours, source)
        VALUES
            (%(code_dept)s, %(semaine)s, %(syndrome)s, %
        (nb_passages_urg)s,
             %(nb_consultations)s, %(nb_total_passages)s, %
        (taux_recours)s, %(source)s)
        ON CONFLICT (code_dept, semaine, syndrome, source)
        DO UPDATE SET
            nb_passages_urg   = EXCLUDED.nb_passages_urg,
            nb_consultations  = EXCLUDED.nb_consultations,
            nb_total_passages = EXCLUDED.nb_total_passages,
            taux_recours      = EXCLUDED.taux_recours,
            updated_at        = CURRENT_TIMESTAMP;
    """

    sql_indicateurs = """
        INSERT INTO indicateurs_epidemiques
            (code_dept, semaine, syndrome, taux_incidence, z_score,
```

```
            r0_estime, nb_annees_reference, statut)
        VALUES
            (%(code_dept)s, %(semaine)s, %(syndrome)s, %
        (taux_incidence)s,
             %(z_score)s, %(r0_estime)s, %(nb_annees_reference)s, %
        (statut)s)
        ON CONFLICT (code_dept, semaine, syndrome)
        DO UPDATE SET
            taux_incidence      = EXCLUDED.taux_incidence,
            z_score             = EXCLUDED.z_score,
            r0_estime           = EXCLUDED.r0_estime,
            nb_annees_reference = EXCLUDED.nb_annees_reference,
            statut              = EXCLUDED.statut,
            updated_at          = CURRENT_TIMESTAMP;
    """

    with hook.get_conn() as conn:
        with conn.cursor() as cur:
            for enreg in donnees_brutes.get("donnees", []):
                cur.execute(sql_donnees, {
                    "code_dept":         enreg.get("code_dept"),
                    "semaine":           enreg.get("semaine"),
                    "syndrome":          enreg.get("syndrome"),
                    "nb_passages_urg":   enreg.get("nb_passages_urg",
        0),
                    "nb_consultations":  enreg.get("nb_consultations",
        0),
                    "nb_total_passages":
        enreg.get("nb_total_passages", 0),
                    "taux_recours":      enreg.get("taux_recours"),
                    "source":            enreg.get("source",
        "COMBINE"),
                })
            for indicateur in indicateurs:
                cur.execute(sql_indicateurs, indicateur)
            conn.commit()
    logger.info(f"{len(indicateurs)} indicateurs insérés/mis à jour
        pour la semaine {semaine}")

inserer_postgres = PythonOperator(
    task_id="inserer_donnees_postgres",
    python_callable=inserer_donnees_postgres,
    templates_dict={
        "semaine": "{{ execution_date.year }}-S{{ '%02d' %
        execution_date.isocalendar()[1] }}",
    },
    provide_context=True,
)
```

Etape 8 — BranchPythonOperator `evaluer_situation_epidemique`

Objectif : Décider du chemin d'exécution selon la situation épidémique observée dans PostgreSQL.

```python
from airflow.operators.python import BranchPythonOperator,
        PythonOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook
import logging

logger = logging.getLogger(__name__)

def evaluer_situation_epidemique(**context) -> str:
    """
    Lit les indicateurs de la semaine depuis PostgreSQL et détermine
        le chemin
    d'exécution selon la situation épidémique la plus sévère observée.
    Returns: task_id de la branche à exécuter
    """
    semaine: str = context["templates_dict"]["semaine"]
    hook = PostgresHook(postgres_conn_id="postgres_ars")

    with hook.get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("""
                SELECT statut,
                       COUNT(DISTINCT code_dept) AS nb_depts,
                       ARRAY_AGG(DISTINCT code_dept) AS depts
                FROM indicateurs_epidemiques
                WHERE semaine = %s
                GROUP BY statut
            """, (semaine,))
            resultats = {row[0]: {"nb": row[1], "depts": row[2]} for
        row in cur.fetchall()}

    nb_urgence = resultats.get("URGENCE", {}).get("nb", 0)
    nb_alerte  = resultats.get("ALERTE", {}).get("nb", 0)
    depts_urgence = resultats.get("URGENCE", {}).get("depts", [])
    depts_alerte  = resultats.get("ALERTE", {}).get("depts", [])

    context["task_instance"].xcom_push(key="nb_urgence",
        value=nb_urgence)
    context["task_instance"].xcom_push(key="nb_alerte",
        value=nb_alerte)
    context["task_instance"].xcom_push(key="depts_urgence",
        value=depts_urgence)
    context["task_instance"].xcom_push(key="depts_alerte",
        value=depts_alerte)

    logger.info(f"Semaine {semaine}: {nb_urgence} depts URGENCE,
        {nb_alerte} depts ALERTE")

    if nb_urgence > 0:
```

```
        return "declencher_alerte_ars"
    elif nb_alerte > 0:
        return "envoyer_bulletin_surveillance"
    else:
        return "confirmer_situation_normale"

def declencher_alerte_ars(**context) -> None:
    depts = context["task_instance"].xcom_pull(
        task_ids="evaluer_situation_epidemique", key="depts_urgence"
    )
    logger.critical(f"ALERTE ARS DÉCLENCHÉE — Départements en URGENCE
        : {depts}")

def envoyer_bulletin_surveillance(**context) -> None:
    depts = context["task_instance"].xcom_pull(
        task_ids="evaluer_situation_epidemique", key="depts_alerte"
    )
    logger.warning(f"Bulletin de surveillance envoyé — Départements en
        ALERTE : {depts}")

def confirmer_situation_normale(**context) -> None:
    logger.info("Situation épidémiologique normale en Occitanie —
        aucune action requise")

evaluer = BranchPythonOperator(
    task_id="evaluer_situation_epidemique",
    python_callable=evaluer_situation_epidemique,
    templates_dict={
        "semaine": "{{ execution_date.year }}-S{{ '%02d' %
        execution_date.isocalendar()[1] }}",
    },
    provide_context=True,
)

alerte_ars   = PythonOperator(task_id="declencher_alerte_ars",
        python_callable=declencher_alerte_ars,
        provide_context=True)
bulletin     = PythonOperator(task_id="envoyer_bulletin_surveillance",
        python_callable=envoyer_bulletin_surveillance,
        provide_context=True)
normale      = PythonOperator(task_id="confirmer_situation_normale",
        python_callable=confirmer_situation_normale,
        provide_context=True)
```

Etape 9 — Implémentation de `generer_rapport_hebdomadaire`

Objectif : Générer un rapport JSON structuré, le sauvegarder dans le volume Docker et l'enregistrer dans PostgreSQL.

Structure JSON attendue :

```json
{
  "semaine": "2024-S04",
  "region": "Occitanie",
  "code_region": "76",
  "date_generation": "2024-01-29T06:42:00.000Z",
  "situation_globale": "ALERTE",
  "nb_departements_surveilles": 13,
  "departements_en_urgence": [],
  "departements_en_alerte": [
    {"code": "34", "nom": "Hérault"},
    {"code": "31", "nom": "Haute-Garonne"}
  ],
  "indicateurs": [
    {
      "code_dept": "34",
      "nom_dept": "Hérault",
      "syndrome": "GRIPPE",
      "taux_incidence_100k": 212.5,
      "z_score": 1.87,
      "r0_estime": 1.23,
      "statut": "ALERTE"
    }
  ],
  "recommandations": [
    "Surveillance renforcée des indicateurs pour les 48h suivantes",
    "Envoi d'un bulletin de surveillance aux partenaires de santé"
  ],
  "genere_par": "ars_epidemio_dag v1.0",
  "pipeline_version": "2.8"
}
```


Implémentation :

```python
from airflow.utils.trigger_rule import TriggerRule
from airflow.operators.python import PythonOperator
from airflow.providers.postgres.hooks.postgres import PostgresHook
import json, os, logging
from datetime import datetime

logger = logging.getLogger(__name__)

def generer_rapport_hebdomadaire(**context) -> None:
    """
    Génère le rapport hebdomadaire de surveillance épidémiologique.
    S'exécute quelle que soit la branche empruntée.
    """
    semaine: str = context["templates_dict"]["semaine"]
    hook = PostgresHook(postgres_conn_id="postgres_ars")

    with hook.get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("""
```

```
                SELECT ie.code_dept, d.nom, ie.syndrome,
        ie.taux_incidence,
                       ie.z_score, ie.r0_estime, ie.statut,
        ie.nb_annees_reference
                FROM indicateurs_epidemiques ie
                JOIN departements d ON ie.code_dept = d.code_dept
                WHERE ie.semaine = %s
                ORDER BY ie.statut DESC, ie.taux_incidence DESC
            """, (semaine,))
            indicateurs = cur.fetchall()

    statuts = [row[6] for row in indicateurs]
    if "URGENCE" in statuts:
        situation_globale = "URGENCE"
    elif "ALERTE" in statuts:
        situation_globale = "ALERTE"
    else:
        situation_globale = "NORMAL"

    depts_urgence = {row[0]: row[1] for row in indicateurs if row[6]
        == "URGENCE"}
    depts_alerte  = {row[0]: row[1] for row in indicateurs if row[6]
        == "ALERTE"}

    recommandations_par_niveau = {
        "URGENCE": [
            "Activation du plan de réponse épidémique régional",
            "Renforcement des équipes de surveillance dans les
        services d'urgences",
            "Communication renforcée auprès des professionnels de
        santé libéraux",
            "Notification immédiate à Santé Publique France et au
        Ministère de la Santé",
        ],
        "ALERTE": [
            "Surveillance renforcée des indicateurs pour les 48h
        suivantes",
            "Envoi d'un bulletin de surveillance aux partenaires de
        santé",
            "Vérification des capacités d'accueil des services
        d'urgences",
        ],
        "NORMAL": [
            "Maintien de la surveillance standard",
            "Prochain point épidémiologique dans 7 jours",
        ],
    }

    rapport = {
        "semaine": semaine,
        "region": "Occitanie",
        "code_region": "76",
        "date_generation": datetime.utcnow().isoformat(),
```

```
        "situation_globale": situation_globale,
        "nb_departements_surveilles": 13,
        "departements_en_urgence": [{"code": c, "nom": n} for c, n in
         depts_urgence.items()],
        "departements_en_alerte":  [{"code": c, "nom": n} for c, n in
         depts_alerte.items()],
        "indicateurs": [
            {
                "code_dept":         row[0],
                "nom_dept":          row[1],
                "syndrome":          row[2],
                "taux_incidence_100k": row[3],
                "z_score":           row[4],
                "r0_estime":         row[5],
                "statut":            row[6],
            }
            for row in indicateurs
        ],
        "recommandations":
         recommandations_par_niveau[situation_globale],
        "genere_par": "ars_epidemio_dag v1.0",
        "pipeline_version": "2.8",
    }

    annee = semaine.split("-")[0]
    num_sem = semaine.split("-")[1]
    local_path =
        f"/data/ars/rapports/{annee}/{num_sem}/rapport_{semaine}.json"
    os.makedirs(os.path.dirname(local_path), exist_ok=True)
    with open(local_path, "w", encoding="utf-8") as f:
        json.dump(rapport, f, ensure_ascii=False, indent=2)

    hook2 = PostgresHook(postgres_conn_id="postgres_ars")
    with hook2.get_conn() as conn:
        with conn.cursor() as cur:
            cur.execute("""
                INSERT INTO rapports_ars
                    (semaine, situation_globale, nb_depts_alerte,
         nb_depts_urgence,
                     rapport_json, chemin_local)
                VALUES (%s, %s, %s, %s, %s, %s)
                ON CONFLICT (semaine) DO UPDATE SET
                    situation_globale = EXCLUDED.situation_globale,
                    nb_depts_alerte   = EXCLUDED.nb_depts_alerte,
                    nb_depts_urgence  = EXCLUDED.nb_depts_urgence,
                    rapport_json      = EXCLUDED.rapport_json,
                    chemin_local      = EXCLUDED.chemin_local,
                    updated_at        = CURRENT_TIMESTAMP
            """, (
                semaine, situation_globale,
                len(depts_alerte), len(depts_urgence),
                json.dumps(rapport, ensure_ascii=False),
```

```
                local_path,
            ))
            conn.commit()

    logger.info(f"Rapport {semaine} généré — Statut :
        {situation_globale}")

generer_rapport = PythonOperator(
    task_id="generer_rapport_hebdomadaire",
    python_callable=generer_rapport_hebdomadaire,
    templates_dict={
        "semaine": "{{ execution_date.year }}-S{{ '%02d' %
        execution_date.isocalendar()[1] }}",
    },
    trigger_rule=TriggerRule.NONE_FAILED_MIN_ONE_SUCCESS,
    provide_context=True,
)
```

Etape 10 — Monitoring et validation finale

Objectif : Vérifier le bon fonctionnement de l'ensemble du pipeline et analyser les métriques.

10.1 Vérification des workers Celery

```
# Vérifier les workers actifs via Flower
# -> http://localhost:5555

# Vérifier les workers via CLI
docker compose exec airflow-worker celery --app
        airflow.executors.celery_executor.app inspect active

# Voir les tâches en cours d'exécution
docker compose exec airflow-worker celery --app
        airflow.executors.celery_executor.app inspect reserved
```

10.2 Consultation des logs

```
# Voir les logs du scheduler
docker compose logs airflow-scheduler --tail=50

# Voir les logs d'un worker en temps réel
docker compose logs -f airflow-worker

# Voir les logs d'un service spécifique
docker compose logs airflow-webserver --tail=20
```

10.3 Analyse de la consommation des ressources

```
# Consommation CPU/RAM de tous les conteneurs
docker stats
```

```
# Vérifier le contenu du volume de données
docker compose exec airflow-worker find /data/ars -type f | sort

# Vérifier la taille du volume
docker compose exec airflow-worker du -sh /data/ars/
```

10.4 Checklist de validation finale

Avant de soumettre votre TP, vérifiez chaque point :

- ☐ Tous les services Docker Compose sont en état `running` (`docker compose ps`)
- ☐ Le DAG `ars_epidemio_dag` est visible dans l'UI et ne présente pas d'erreur d'import
- ☐ Un DAG Run complet s'est terminé avec succès (toutes les tâches en vert)
- ☐ Les 5 tables PostgreSQL existent et contiennent des données
- ☐ Les fichiers bruts sont présents dans `/data/ars/raw/`
- ☐ Le rapport JSON est présent dans `/data/ars/rapports/`
- ☐ Les logs des tâches dans l'UI Airflow sont lisibles et informatifs
- ☐ Le code Python respecte les conventions PEP 8 et utilise des type hints
- ☐ Aucun credential n'est écrit en dur dans le code DAG
- ☐ Le pipeline est entièrement rejouable : relancer sur la même semaine ne produit pas de doublons

---

8. Livrables attendus

Vous devez déposer une archive `<nom_prenom>_tp_note_airflow.zip` contenant la structure suivante :

```
<nom_prenom>_tp_note_airflow/
├── dags/
│   ├── ars_epidemio_dag.py        # DAG principal complet
│   └── sql/
│       └── init_ars_epidemio.sql  # Schéma SQL complet (CREATE TABLE
+ INSERT)
├── scripts/
│   ├── collecte_sursaud.py        # Script de collecte
│   └── calcul_indicateurs.py      # Script de calcul des indicateurs
├── docker-compose.yaml
├── .env.example                   # Fichier .env sans valeurs
sensibles
├── screenshots/
│   ├── 01_docker_compose_ps.png        # Tous les services en état
Running
│   ├── 02_airflow_ui_dag_overview.png  # Vue du DAG dans l'UI
│   ├── 03_dag_run_success.png          # DAG Run terminé avec succès
│   ├── 04_postgres_tables_remplies.png # Les 5 tables avec des
données
│   ├── 05_volume_arborescence.png      # Arborescence /data/ars/
│   └── 06_flower_workers.png           # Workers Celery actifs dans
Flower
```

```
├── output/
│   └── rapport_<semaine>.json          # Exemple de rapport JSON
généré
└── README.md                           # Documentation selon le
format ci-dessous
```

Format du README.md :

```
# TP Noté Airflow — Data Platform Santé Publique ARS Occitanie

## Auteur
- Nom : [votre nom]
- Prénom : [votre prénom]
- Formation : [votre formation]
- Date : [date du TP]

## Prérequis
- Docker Desktop >= 4.0
- Docker Compose >= 2.0
- Python >= 3.11 (pour les tests locaux)

## Instructions de déploiement

### 1. Démarrage de la stack
[commandes détaillées]

### 2. Configuration des connexions et variables Airflow
[procédure pas à pas]

### 3. Démarrage du pipeline
[procédure : activer le DAG, déclencher un Run manuel]

## Architecture des données
[brève description des choix de partitionnement et du schéma
        PostgreSQL]

## Décisions techniques
[justification des choix effectués : image Docker utilisée, gestion du
        réseau, etc.]

## Difficultés rencontrées et solutions
[honnêteté attendue sur les difficultés et comment elles ont été
        résolues]
```

---

9. Contraintes techniques obligatoires

Les contraintes suivantes sont **non négociables** :

1. **Aucun credential en dur dans le code DAG** — toutes les connexions passent par les Connections Airflow ou les Variables Airflow, jamais via des chaînes de caractères en clair dans `ars_epidemio_dag.py`

2. **Idempotence obligatoire** — relancer le pipeline sur la même semaine ne doit pas créer de doublons dans PostgreSQL (utiliser `ON CONFLICT DO NOTHING` ou `ON CONFLICT DO UPDATE`)

3. **Type hints Python** sur toutes les fonctions Python définies dans le DAG (`def ma_fonction(param: type) -> type:`)

4. **Logging structuré** — utiliser `logging.getLogger(__name__)` pour les messages applicatifs ; les `print()` sont tolérés uniquement pour les sorties lues par Airflow (`print("COLLECTE_OK:...")`)

5. **`max_active_runs=1`** sur le DAG — pour éviter les conditions de course sur les données partagées (volume Docker et tables PostgreSQL)

6. **`catchup=True`** sur le DAG — le pipeline doit pouvoir rejouer les semaines passées ; cette fonctionnalité est au cœur de la valeur d'Apache Airflow

7. **Gestion explicite des exceptions** dans tous les scripts Python — un script qui échoue silencieusement est inacceptable en production

---

10. Ressources autorisées

Pendant la durée du TP, **seules** les ressources suivantes sont autorisées :

Documentation officielle

Outil / Bibliothèque	URL
Apache Airflow 2.8	https://airflow.apache.org/docs/apache-airflow/2.8.0/
Docker Compose	https://docs.docker.com/compose/
PostgreSQL	https://www.postgresql.org/docs/13/
psycopg2	https://www.psycopg.org/docs/
NumPy	https://numpy.org/doc/stable/
SciPy	https://docs.scipy.org/doc/scipy/

Données Open Data

Source	URL
IAS® Syndrome Grippal — data.gouv.fr	https://www.data.gouv.fr/fr/datasets/indicateur-avance-sanitaire-ias-syndrome-grippal/
IAS® Gastro-Entérite — data.gouv.fr	https://www.data.gouv.fr/fr/datasets/indicateur-avance-sanitaire-ias-gastro-enterite/

Source	URL
CSV Grippal (téléchargement direct)	https://www.data.gouv.fr/api/1/datasets/r/35f46fbb-7a97-46b3-a93c-35a471033447
CSV Gastro (téléchargement direct)	https://www.data.gouv.fr/api/1/datasets/r/6c415be9-4ebf-4af5-b0dc-9867bb1ec0e3
Open-Meteo (bonus uniquement)	https://archive-api.open-meteo.com/v1/archive

Ressources strictement interdites

- Tout outil d'IA générative : ChatGPT, GitHub Copilot, Claude, Gemini, Mistral, etc.
- Stack Overflow, Reddit, forums techniques, blogs
- Code d'autres étudiants
- Tout code non produit par vous-même pendant la séance

En cas de doute sur la légitimité d'une ressource, posez la question à l'examinateur avant de l'utiliser.

Document rédigé pour la formation Data Engineering — IPSSI Montpellier Module Orchestration avec Apache Airflow — Jour 4 — TP Noté — Durée : 4 heures — Travail individuel